Experiment No.	1
Experiment Title	Universal Exploratory Data Analysis, Data Preprocessing, and Feature Engineering across Python Scientific Packages [1]
Student Name	Kavinsoorya K S
Register Number	3122247001030
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	July 25, 2026

1 Objective

The objective of this experiment is to explore Python scientific computing and data analysis libraries (`NumPy`, `Pandas`, `SciPy`, `Scikit-Learn`, `Matplotlib`, and `Seaborn`) to perform automated Exploratory Data Analysis (EDA), feature selection, data preprocessing, baseline model fitting, and comparative performance evaluation across five benchmark machine learning datasets.

2 Problem Statement

In machine learning workflows, raw datasets differ significantly in feature dimensionality, scale, data types, missingness, and problem domains (tabular classification, continuous regression, unstructured text mining, medical diagnosis, and computer vision). To establish robust predictive models, an automated data processing and analytical pipeline must be developed to identify task types, inspect distributions, impute noise, extract relevant features, partition datasets into train-test splits, fit baseline machine learning algorithms, and quantify evaluation metrics.

3 Universal EDA Architecture (`perform_EDA`)

To standardize exploratory analysis across diverse data formats, a reusable Python function `perform_EDA()` was constructed in `universal_eda.ipynb`. The complete code implementation is provided below:

Listing 1: Complete Universal EDA Implementation Code (perform_EDA)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

def perform_eda(file_path):
    """
    Universal EDA (Stops before preprocessing/modeling)

    Works for:
    Iris, Loan, Diabetes, MNIST(CSV), Spam(SMS)

```

```
    Usage:
        perform_eda("file.csv")
    """

    try:
        df = pd.read_csv(file_path, encoding="latin1")
    except Exception:
        df = pd.read_csv(file_path)

    print("="*80)
    print("DATASET OVERVIEW")
    print("="*80)
    print("Shape :", df.shape)
    print("\nColumns\n", df.columns.tolist())
    print("\nInfo")
    df.info()
    print("\nFirst 5 Rows")
    print(df.head())
    print("\nSummary")
    print(df.describe(include="all"))
    print("\nMissing Values")
    print(df.isnull().sum())
    print("\nDuplicate Rows :", df.duplicated().sum())

    # ---------- Spam ----------
    if "v1" in df.columns and "v2" in df.columns:
        df = df.drop(columns=[c for c in df.columns if "Unnamed"
           in c], errors="ignore")
        df.columns = ["Label", "Message"]
        df["Length"] = df["Message"].str.len()

        #1 Countplot
        sns.countplot(data=df, x="Label")
        plt.title("Spam vs Ham")
        plt.show()

        #2 Message Length Distribution
        sns.histplot(df["Length"], kde=True)
        plt.title("Message Length Distribution")
        plt.show()

        #3 Boxplot
        sns.boxplot(data=df, x="Label", y="Length")
        plt.title("Message Length Boxplot")
        plt.show()

        #4 Violin Plot
        sns.violinplot(data=df, x="Label", y="Length")
        plt.title("Message Length Violin Plot")
        plt.show()

```

```
        #5 QQ Plot
        stats.probplot(df["Length"], dist="norm", plot=plt)
        plt.title("QQ Plot")
        plt.show()

        #6 Scatter Plot
        plt.scatter(range(len(df)), df["Length"], s=8)
        plt.title("Length Scatter")
        plt.xlabel("Message Index")
        plt.ylabel("Length")
        plt.show()

        #7 Groupby summary
        print(df.groupby("Label")["Length"].describe())
        return

    # ---------- MNIST ----------
    if "label" in df.columns and df.shape[1] == 785:

        #1 Digit Distribution Countplot
        sns.countplot(data=df, x="label")
        plt.title("Digit Distribution")
        plt.show()

        #2 Sample Grayscale Images
        plt.figure(figsize=(12, 5))
        for i in range(10):
            plt.subplot(2, 5, i+1)
            plt.imshow(df.iloc[i, 1:].values.reshape(28, 28),
                cmap="gray")
            plt.title(df.iloc[i, 0])
            plt.axis("off")
        plt.tight_layout()
        plt.show()

        pixels = df.iloc[:, 1:].values.flatten()

        #3 Pixel Distribution
        sns.histplot(pixels, bins=40)
        plt.title("Pixel Distribution")
        plt.show()

        #4 Pixel Boxplot
        sns.boxplot(x=pixels)
        plt.title("Pixel Boxplot")
        plt.show()

        #5 Pixel Violin Plot
        sns.violinplot(x=pixels)
        plt.title("Pixel Violin Plot")
        plt.show()
```

```
114
115         #6 Pixel QQ Plot
116         stats.probplot(pixels[:5000], dist="norm", plot=plt)
117         plt.title("Pixel QQ Plot")
118         plt.show()
119
120         #7 Value counts
121         print(df["label"].value_counts())
122         return
123
124     # ---------- Diabetes Audit ----------
125     if "Outcome" in df.columns:
126         cols = ["Glucose", "BloodPressure", "SkinThickness", "
               Insulin", "BMI"]
127         print("\nPossible Missing Values Stored as Zero")
128         for c in cols:
129             if c in df.columns:
130                 print(f"{c}: {(df[c]==0).sum()}")
131
132     num = df.select_dtypes(include=np.number).columns.tolist()
133     cat = df.select_dtypes(exclude=np.number).columns.tolist()
134
135     #1 Histograms
136     for c in num:
137         sns.histplot(df[c], kde=True, bins=30)
138         plt.title(f"Histogram - {c}")
139         plt.show()
140
141     #2 Boxplots
142     for c in num:
143         sns.boxplot(x=df[c])
144         plt.title(f"Boxplot - {c}")
145         plt.show()
146
147     #3 Violin Plots
148     for c in num:
149         sns.violinplot(x=df[c])
150         plt.title(f"Violin - {c}")
151         plt.show()
152
153     #4 Countplots for Categorical
154     for c in cat:
155         sns.countplot(data=df, x=c)
156         plt.xticks(rotation=45)
157         plt.title(f"Countplot - {c}")
158         plt.tight_layout()
159         plt.show()
160
161     #5 Correlation Heatmap
162     if len(num) > 1:
163         sns.heatmap(df[num].corr(), annot=True, cmap="coolwarm",
```

```
            fmt=".2f")
164         plt.title("Correlation Heatmap")
165         plt.show()
166
167     #6 Pairplots
168     if len(num) <= 8:
169         hue = None
170         for h in ["Species", "species", "Loan_Status", "Outcome"
                ]:
171             if h in df.columns:
172                 hue = h
173                 break
174         sns.pairplot(df, hue=hue)
175         plt.show()
176
177     #7 Normal Probability (QQ) Plots
178     for c in num:
179         stats.probplot(df[c].dropna(), dist="norm", plot=plt)
180         plt.title(f"QQ Plot - {c}")
181         plt.show()
182
183     print("\nEDA Completed.")
```

4 Case Study 1: Iris Dataset

4.1 Dataset Description & ML Task

The Iris dataset contains morphometric measurements of three species of iris flowers (*Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*). The objective is a **Multiclass Classification** task to classify flower species based on four continuous physical attributes.

4.2 Dataset Characteristics & Summary Statistics

- **Total Samples:** 150 instances (50 per species, perfectly balanced).
- **Feature Dimensionality:** 4 numerical predictor features (`SepalLength`, `SepalWidth`, `PedalLength`, `PedalWidth`) and 1 target class (`Species`).
- **Missing Values:** 0 null values across all attributes.
- **Summary Statistics:** `SepalLength` (Mean: 5.84 cm, Std: 0.83), `SepalWidth` (Mean: 3.05 cm, Std: 0.43), `PedalLength` (Mean: 3.76 cm, Std: 1.76), `PedalWidth` (Mean: 1.20 cm, Std: 0.76).

4.3 Exploratory Data Analysis

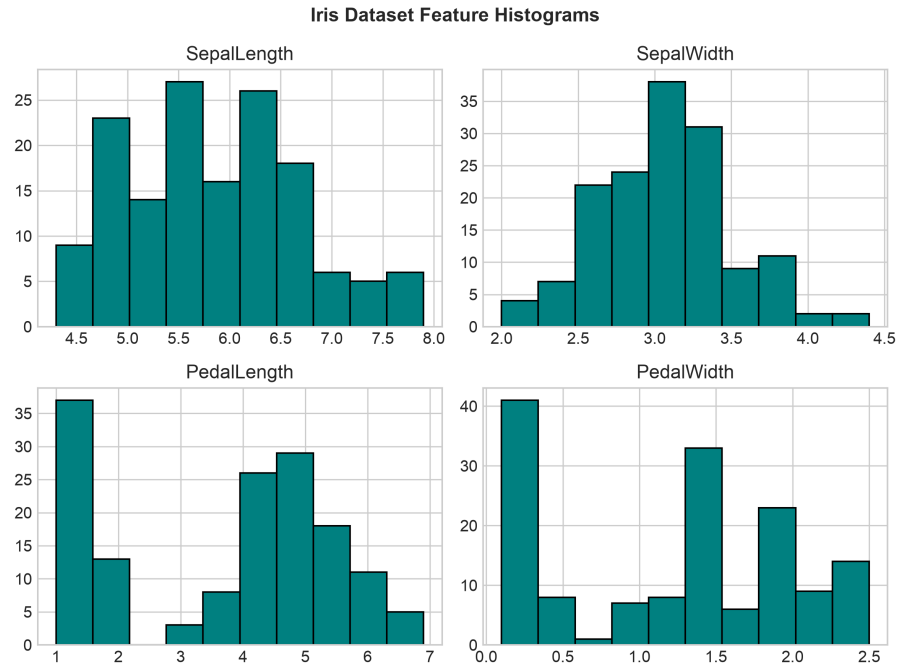

Figure 1: Histograms of Iris Physical Measurements

Inference: `PedalLength` and `PedalWidth` display prominent bimodal distributions, separating *Iris-setosa* from the remaining species, whereas `SepalLength` and `SepalWidth` follow normal bell-shaped curves.

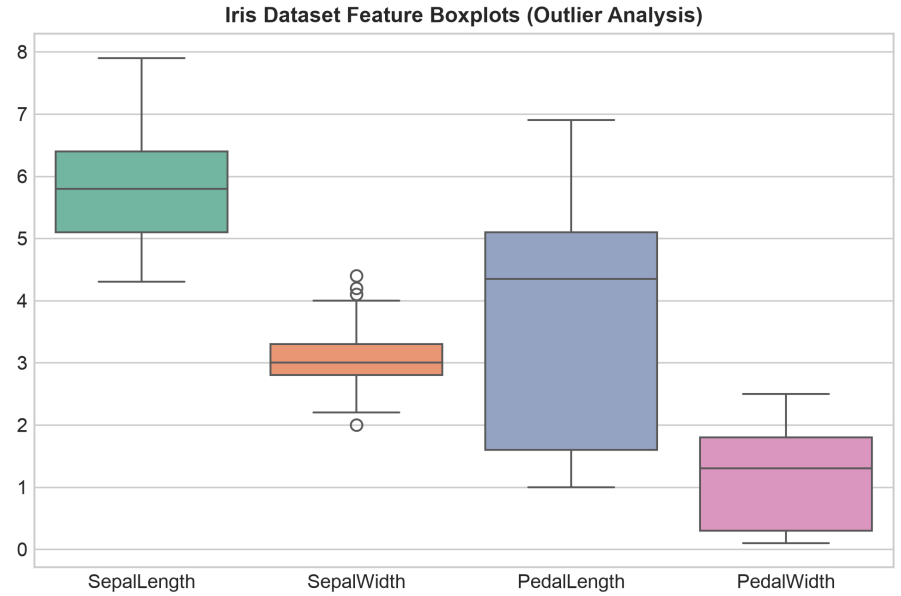

Figure 2: Feature Boxplots for Outlier Inspection

Inference: Boxplots confirm that `PedalLength` and `PedalWidth` possess zero extreme outliers. Minor upper and lower outliers exist in `SepalWidth` around values < 2.0 cm and > 4.0 cm.

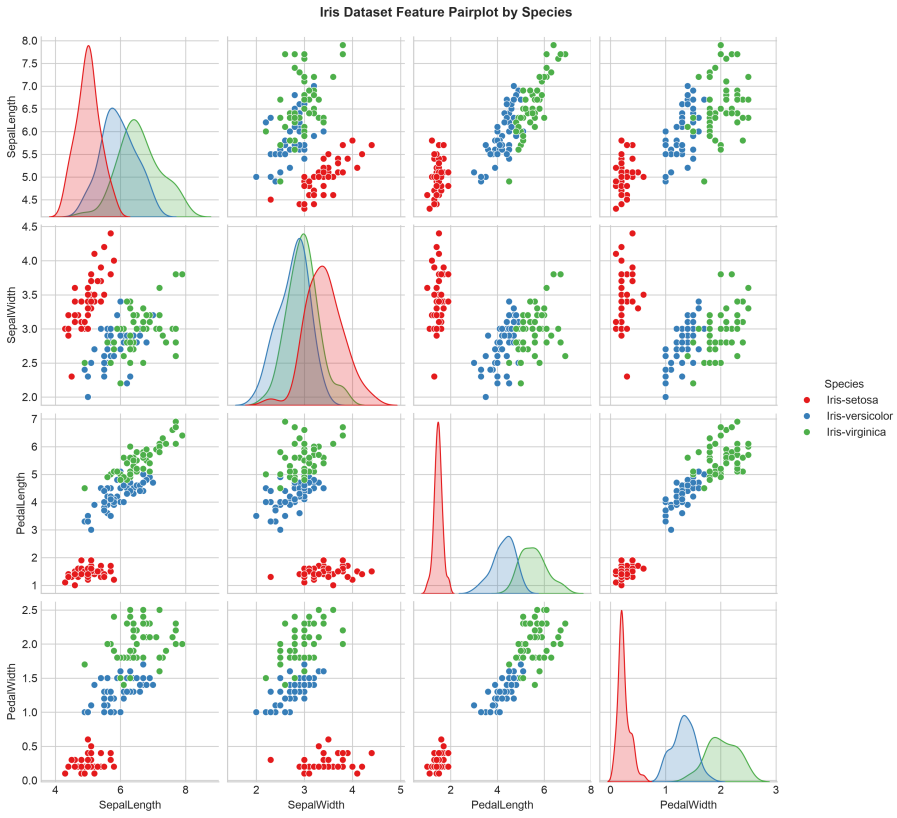

Figure 3: Pairplot of Iris Features Segmented by Species

Inference: *Iris-setosa* is completely linearly separable from *Iris-versicolor* and *Iris-virginica* when plotting `PedalLength` versus `PedalWidth`. Moderate overlap exists between versicolor and virginica.

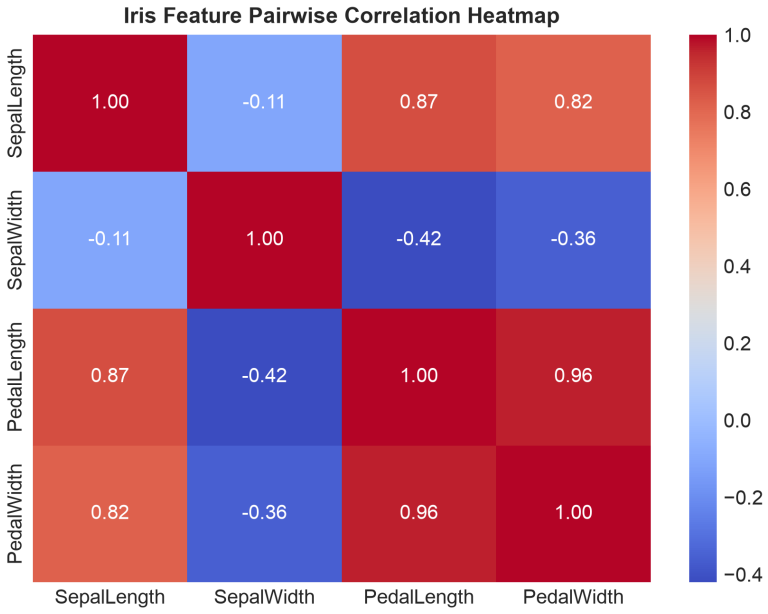

Figure 4: Pairwise Correlation Heatmap for Iris Features

Inference: Strong positive correlation exists between `PedalLength` and `PedalWidth`

($r = 0.96$), as well as between `SepalLength` and `PedalLength` ($r = 0.87$).

4.4 Data Preprocessing & Model Evaluation

- **Train-Test Split:** Partitioned 80% training ($n = 120$) and 20% testing ($n = 30$).
- **Model Trained:** `DecisionTreeClassifier(random_state=42)`.
- **Accuracy Score: 1.0000** (100.0% Test Accuracy).

Figure 5: Decision Tree Confusion Matrix on Iris Test Set

Inference: The confusion matrix shows zero classification errors: 10 setosa, 9 versicolor, and 11 virginica test samples were correctly predicted.

5 Case Study 2: Loan Amount Prediction

5.1 Dataset Description & ML Task

The Loan Amount Prediction dataset contains applicant demographic details, employment status, credit history, and requested financial amounts. The target variable `LoanAmount` is continuous, formulating a **Regression** task.

5.2 Dataset Characteristics & Summary Statistics

- **Sample Count:** 614 initial rows. Dropped 22 rows missing `LoanAmount`, yielding 592 clean records.
- **Features:** 12 predictor features (4 numerical, 7 categorical, 1 identifier dropped).
- **Missing Values Imputed:** `Gender` (13), `Married` (3), `Dependents` (15), `Self_Employed` (32), `Loan_Amount_Term` (14), `Credit_History` (50).
- **Summary Statistics:** `ApplicantIncome` (Mean: \$5,405.54, Max: \$81,000), `CoapplicantIncome` (Mean: \$1,621.27), `LoanAmount` (Mean: \$146.41k, Median: \$128.00k).

5.3 Exploratory Data Analysis

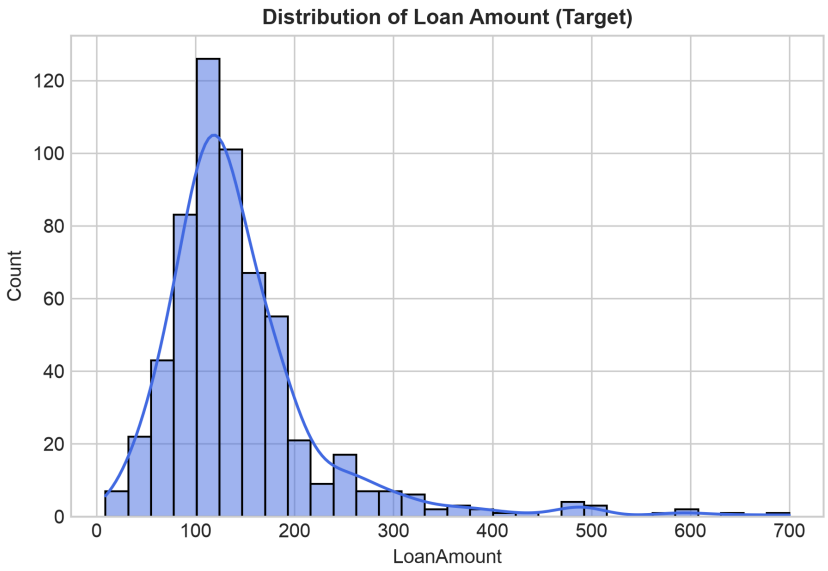

Figure 6: Distribution of Loan Amount (Target Variable)

Inference: The target `LoanAmount` displays a right-skewed log-normal distribution with long upper tails, justifying natural log transformation ($\ln(1+y)$) to stabilize variance.

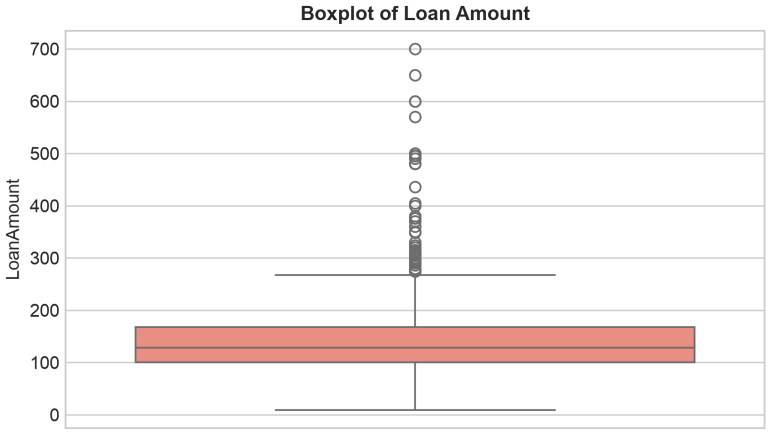

Figure 7: Boxplot of Loan Amount for Outlier Detection

Inference: Numerous extreme upper outliers exist above $300,000, confirming that applicant requests span diverse credit brackets.

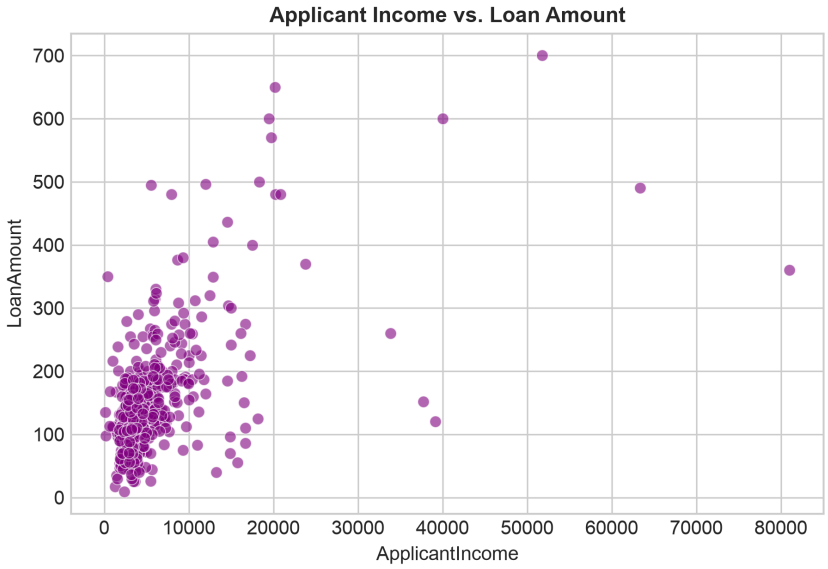

Figure 8: Applicant Income vs. Loan Amount Scatter Plot

Inference: A positive linear relation is present between income and loan size, with variance expanding at higher income levels.

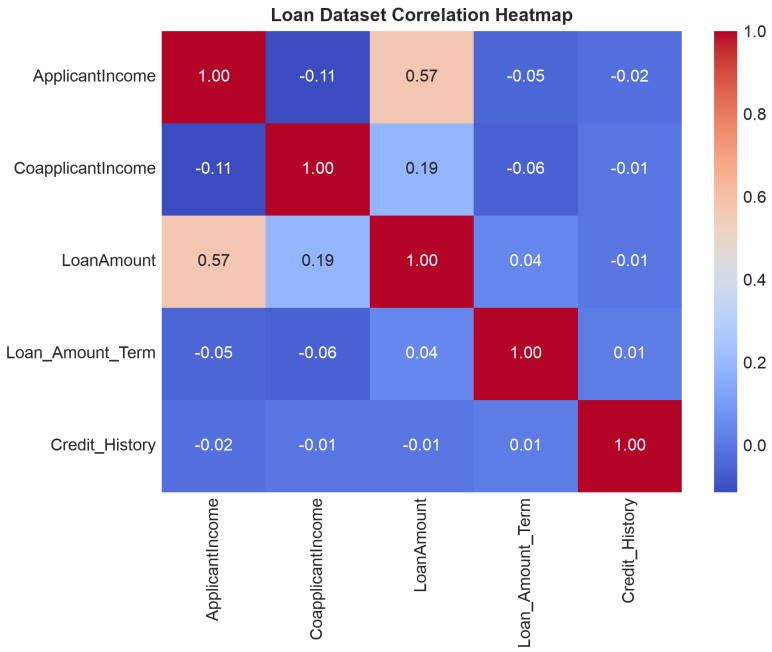

Figure 9: Correlation Heatmap of Numerical Features

Inference: `ApplicantIncome` correlates positively with `LoanAmount` ($r = 0.57$).

5.4 Data Preprocessing & Model Evaluation

- **Preprocessing:** Categorical missing values imputed with column mode; numerical missing values imputed with column median. One-Hot Encoding applied to categorical variables (`drop_first=True`). Target transformed via `np.log1p(LoanAmount)`.

- **Model Trained:** `RandomForestRegressor(n_estimators=200, random_state=42)`.
- **Regression Performance Metrics:**
  - Mean Absolute Error (MAE): **0.2529**
  - Root Mean Squared Error (RMSE): **0.3604**
  - R^2 Score: **0.3228**

6 Case Study 3: Classification of Email Spam

6.1 Dataset Description & ML Task

The SMS Spam Collection dataset contains text messages tagged as `ham` (legitimate) or `spam`. The objective is a **Binary Text Classification** task using Natural Language Processing (NLP).

6.2 Dataset Characteristics & Summary Statistics

- **Total Instances:** 5,572 text messages.
- **Class Distribution:** 4,825 `ham` (86.6%) vs. 747 `spam` (13.4%) instances.
- **Message Length Stats:** Ham length (Mean: 71.0 characters, Median: 52.0), Spam length (Mean: 138.9 characters, Median: 149.0).

6.3 Exploratory Data Analysis

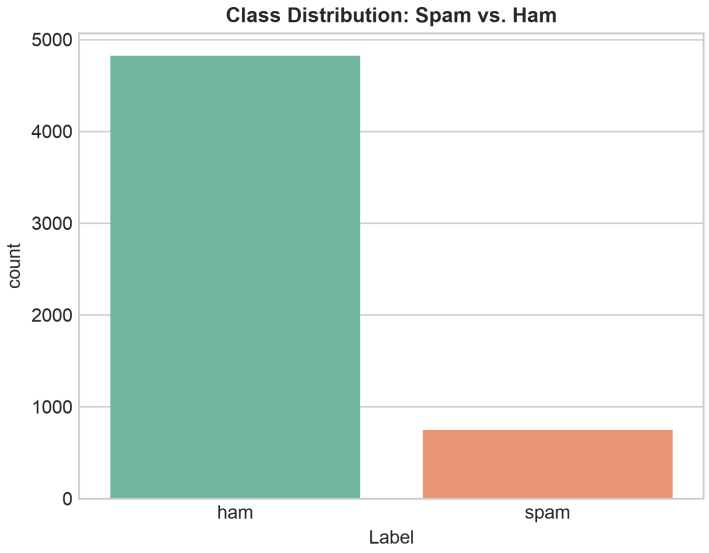

Figure 10: Class Countplot: Spam vs. Ham Messages

Inference: The dataset exhibits severe class imbalance, with legitimate messages outnumbering spam messages by more than 6:1.

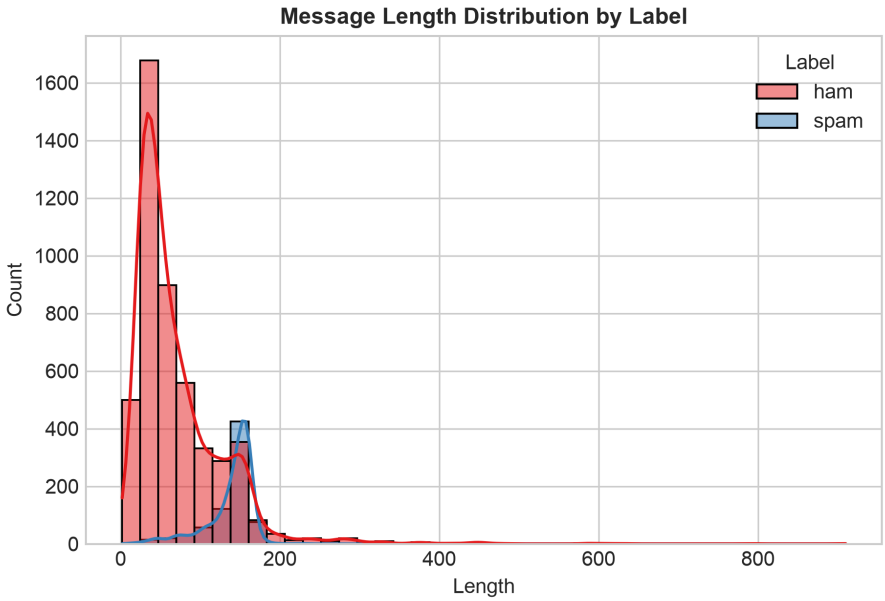

Figure 11: Histogram of Message Character Length by Label

Inference: Spam messages cluster heavily around 130–160 characters (nearing SMS carrier limits), whereas ham messages are generally concise (< 80 characters).

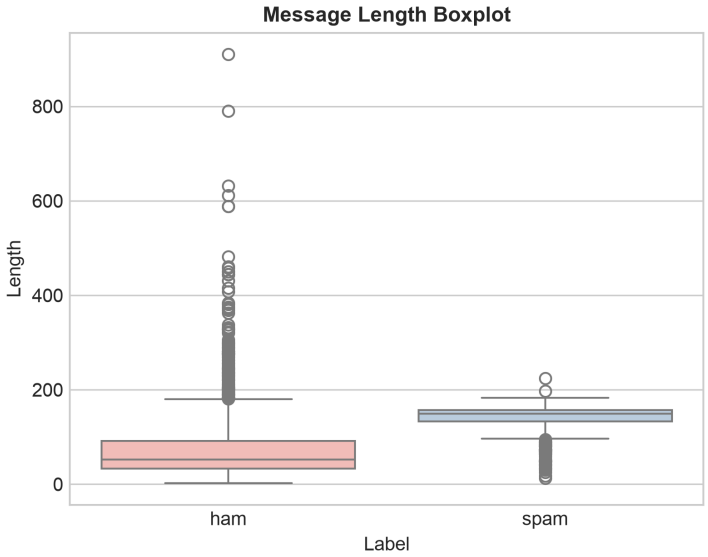

Figure 12: Boxplot of Message Length across Classes

Inference: Boxplots confirm that message length provides strong linear discriminative power between spam and ham messages.

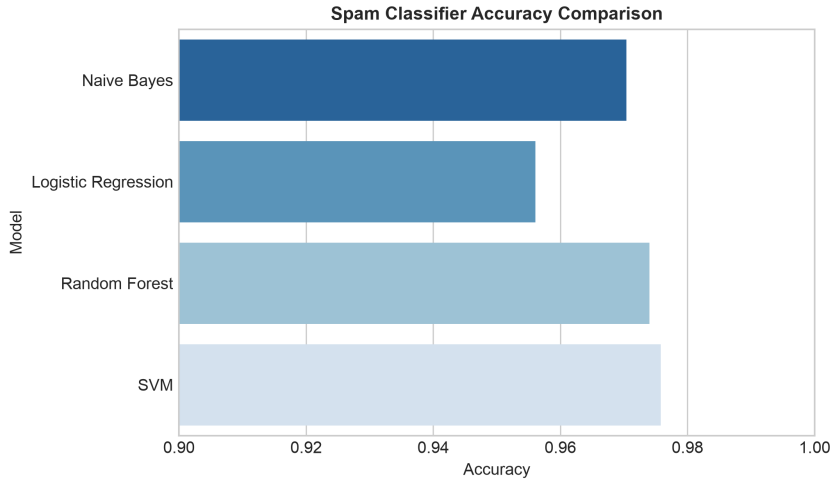

Figure 13: Accuracy Comparison of Text Classification Models

Inference: Support Vector Machines (SVM) achieved the highest overall accuracy of **97.58%**, slightly outperforming Random Forest (97.40%) and Naive Bayes (97.04%).

6.4 Data Preprocessing & Model Evaluation

- **Feature Extraction:** Converted raw text into TF-IDF numerical matrices (`TfidfVectorizer(ma`
- **Model Performance Summary:**
  - Multinomial Naive Bayes: Accuracy = **0.9704**
  - Logistic Regression: Accuracy = **0.9561**
  - Random Forest Classifier: Accuracy = **0.9740**
  - **Support Vector Machine (SVM): Accuracy = 0.9758**

7 Case Study 4: Predicting Diabetes

7.1 Dataset Description & ML Task

The Pima Indians Diabetes dataset contains diagnostic medical measurements from female patients. The task is a **Binary Medical Classification** to predict whether a patient has diabetes (`Outcome = 1`).

7.2 Dataset Characteristics & Summary Statistics

- **Total Instances:** 768 patient records.
- **Class Breakdown:** 500 Non-diabetic (`Outcome = 0`) vs. 268 Diabetic (`Outcome = 1`).
- **Physiologically Invalid Zero Values:** `Glucose` (5 zeros), `BloodPressure` (35 zeros), `SkinThickness` (227 zeros), `Insulin` (374 zeros), `BMI` (11 zeros).

7.3 Exploratory Data Analysis

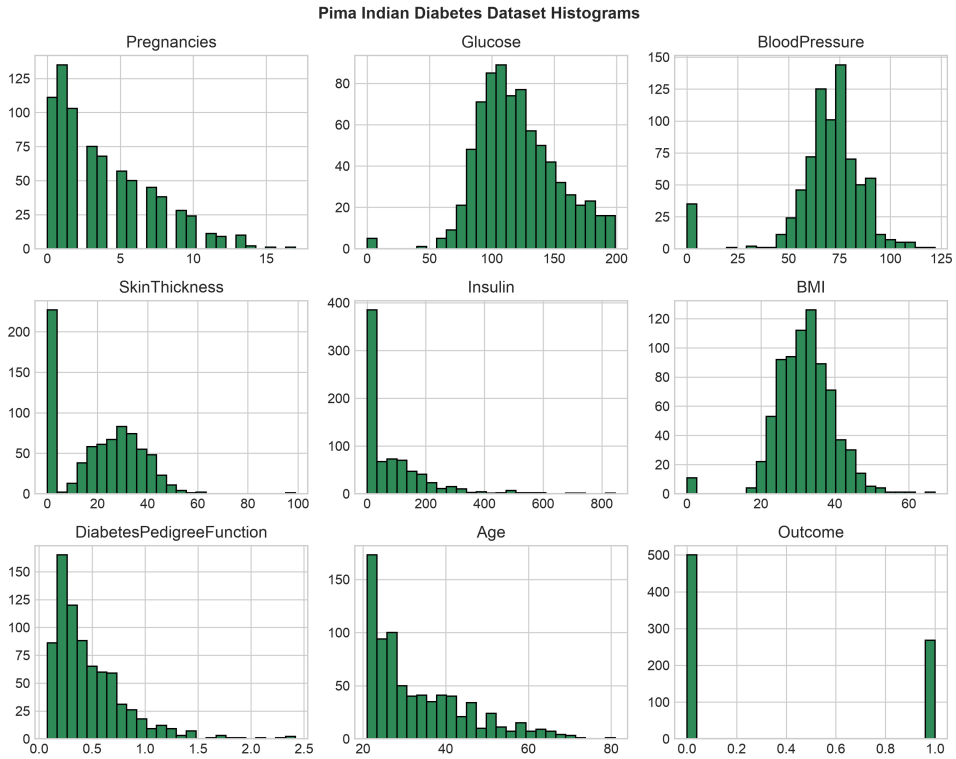

Figure 14: Histograms of Diabetes Clinical Features

Inference: Features like `Insulin` and `DiabetesPedigreeFunction` exhibit sharp positive skewness, while `Glucose` follows a normal distribution centered near 120 mg/dL.

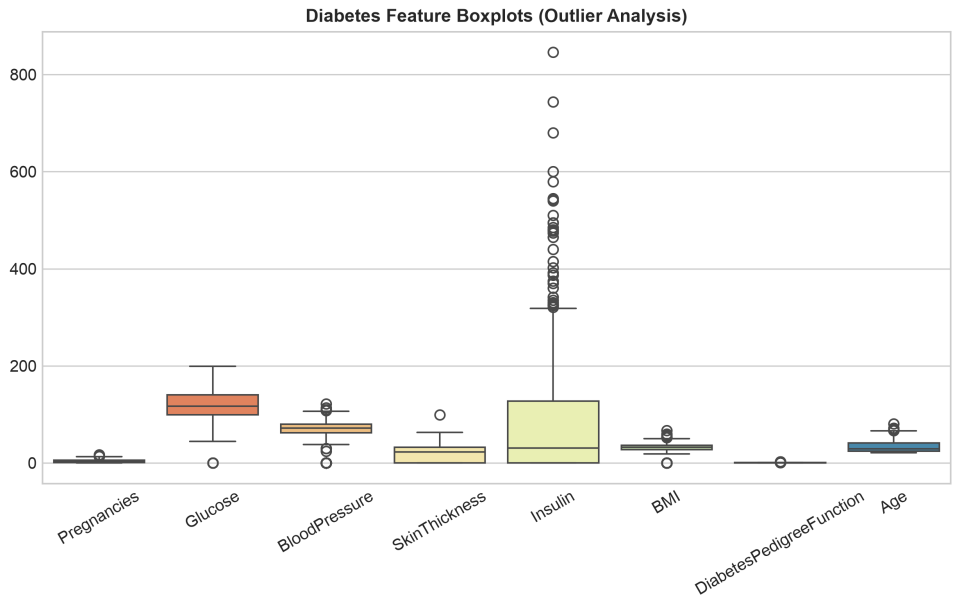

Figure 15: Boxplots of Diabetes Predictor Features

Inference: Outlier analysis indicates high upper values in `Insulin` (> 400) and `Glucose`. Imputing zeros with column medians prevents distance distortions.

Figure 16: Correlation Heatmap of Diabetes Features

Inference: `Glucose` displays the strongest positive correlation with `Outcome` ($r = 0.47$), followed by `BMI` ($r = 0.29$) and `Age` ($r = 0.24$).

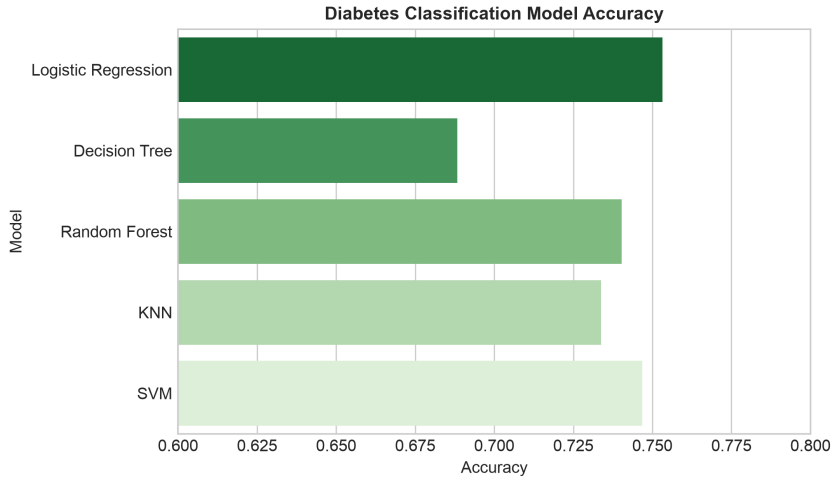

Figure 17: Model Accuracy Comparison for Diabetes Prediction

Inference: Logistic Regression achieved the highest test accuracy of **75.32%**, closely followed by SVM (74.68%) and Random Forest (74.03%).

7.4 Data Preprocessing & Model Evaluation

- **Preprocessing:** Replaced invalid zero values with median column statistics. Scaled features using `StandardScaler`.

- **Model Accuracy Breakdown:**
  - **Logistic Regression: 0.7532** (75.32%)
  - Support Vector Machine (SVM): **0.7468** (74.68%)
  - Random Forest: **0.7403** (74.03%)
  - K-Nearest Neighbors (KNN): **0.7338** (73.38%)
  - Decision Tree Classifier: **0.6883** (68.83%)

8 Case Study 5: Handwritten Character Recognition (MNIST)

8.1 Dataset Description & ML Task

The MNIST subset dataset contains 28×28 pixel grayscale images of handwritten digits (0 through 9). The task is a **Multiclass Image Classification** problem in computer vision.

8.2 Dataset Characteristics & Summary Statistics

- **Sample Count:** 10,000 image instances.
- **Feature Space:** 784 numerical pixel attributes (`1x1` to `28x28`) representing intensity values from 0 (black background) to 255 (white foreground).
- **Class Distribution:** Balanced across digits 0–9 ($\sim 1,000$ samples per digit class).

8.3 Exploratory Data Analysis

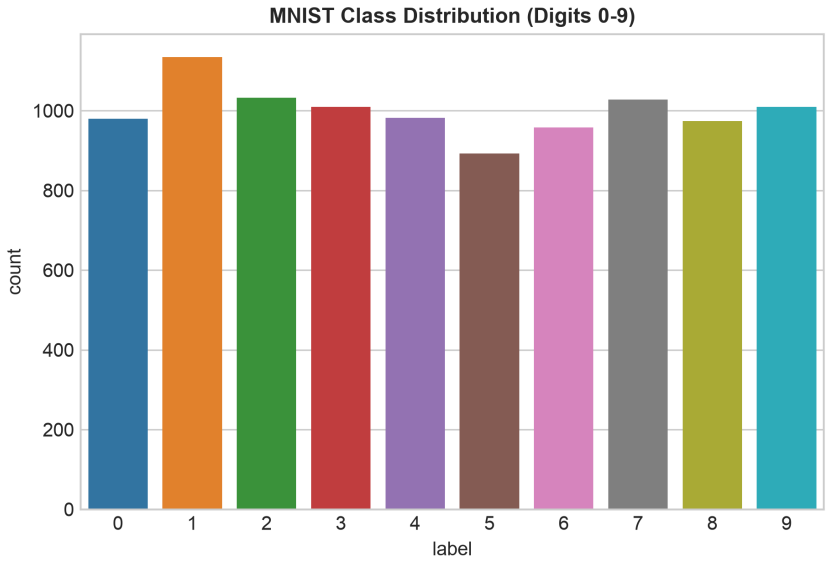

Figure 18: Digit Label Distribution in MNIST Subsample

Inference: The dataset is well-balanced across all 10 digit classes, avoiding majority-class bias during model training.

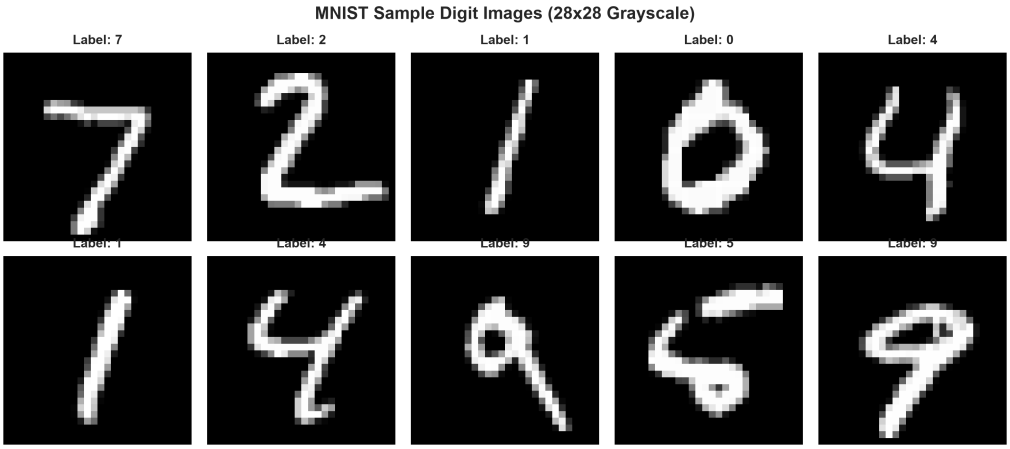

Figure 19: Sample Reconstructed 28×28 Grayscale Digit Images

Inference: Reshaping pixel rows into 28×28 spatial grids recovers recognizable digit shapes, validating feature structure.

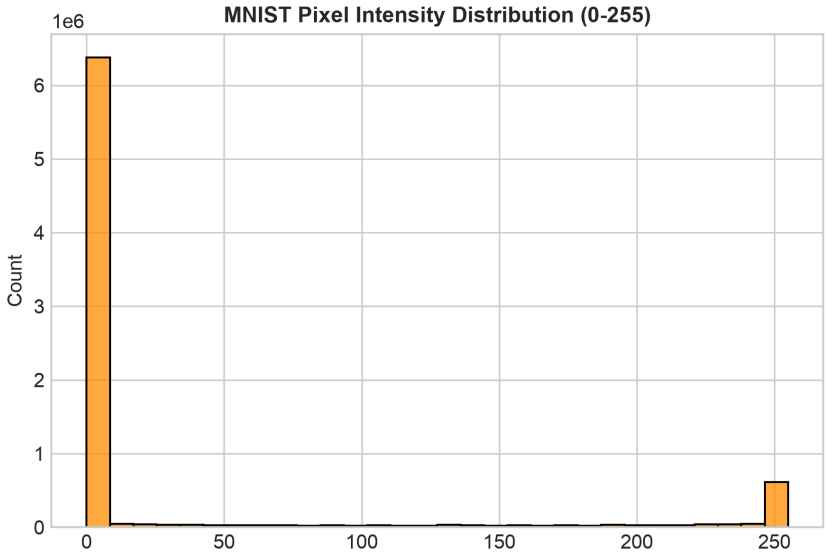

Figure 20: Distribution of Pixel Intensity Values

Inference: Pixel values exhibit a bimodal concentration at 0 (background) and 255 (stroke pixels), confirming the benefit of Min-Max normalization ($X/255.0$).

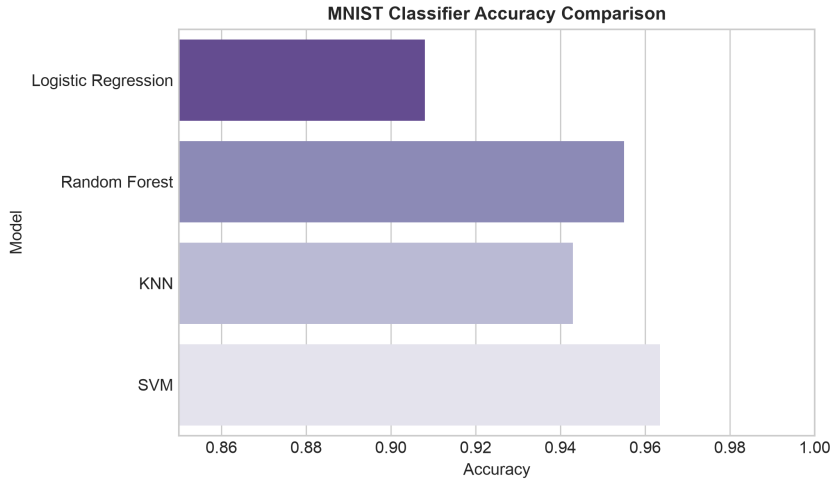

Figure 21: Accuracy Comparison of Vision Classification Models

Inference: Support Vector Machine (RBF Kernel) achieved the top accuracy of **96.35%**, outperforming Random Forest (95.50%) and KNN (94.30%).

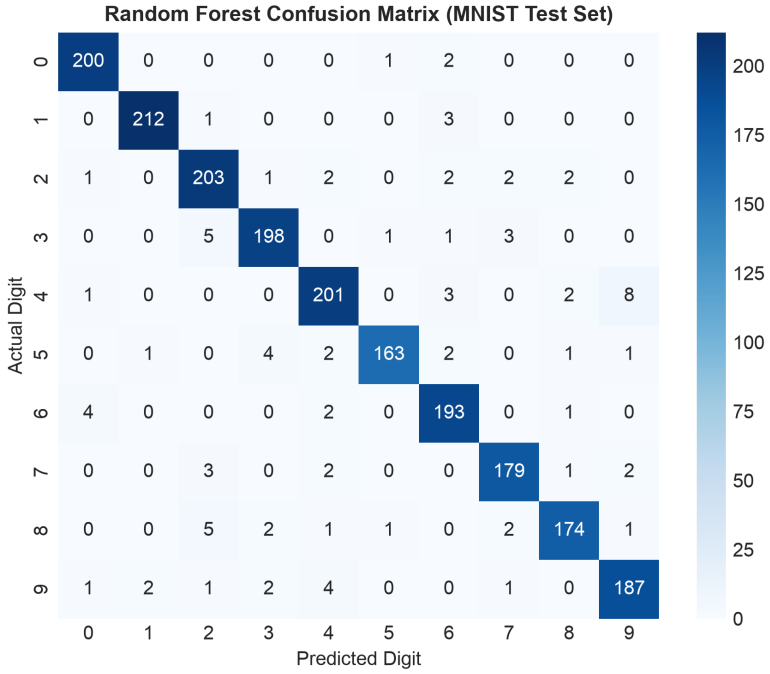

Figure 22: Random Forest Confusion Matrix on MNIST Test Set

Inference: The confusion matrix demonstrates strong diagonal concentration with minor confusion between visually similar digit pairs (e.g., 4 vs. 9 and 3 vs. 8).

8.4 Data Preprocessing & Model Evaluation

- **Preprocessing:** Min-Max scaling ($X = X/255.0$). 80-20 train-test split ($n_{\text{train}} = 8000, n_{\text{test}} = 2000$).
- **Model Accuracy Results:**

- **Support Vector Machine (SVM): 0.9635** (96.35%)
- Random Forest Classifier: **0.9550** (95.50%)
- K-Nearest Neighbors (KNN): **0.9430** (94.30%)
- Logistic Regression: **0.9080** (90.80%)
- Random Forest 5-Fold Cross Validation Mean: **0.9461** (94.61%)

9 Comparative Benchmark Summary

Table 1 summarizes the machine learning task type, feature selection methodology, best algorithm, and achieved performance metric across all five datasets as required by the experiment manual.

Table 1: Comprehensive Comparison of Machine Learning Tasks and Performance

Dataset	**ML Task Type**	**Feature Engineering Technique**	**Best Performing Model**	**Primary Metric**
Iris Dataset	Multiclass Classification	Correlation / Pairwise Scatter Selection	Decision Tree Classifier	Accuracy: **100.0%**
Loan Amount Prediction	Regression Analysis	Mode/Median Impute, One-Hot, Log Transform	Random Forest Regressor	R^2: **0.3228** (RMSE: 0.36)
Classification of Email Spam	Binary Text Classification	String Length, TF-IDF Vectorization ($K = 3000$)	Support Vector Machine (SVM)	Accuracy: **97.58%**
Predicting Diabetes	Binary Medical Classification	Physiological Zero Imputation, StandardScaler	Logistic Regression	Accuracy: **75.32%**
Handwritten Character (MNIST)	Multiclass Vision Classification	Min-Max Pixel Scaling ($X/255$), 2D Reshaping	Support Vector Machine (SVM)	Accuracy: **96.35%**

10 Learning Outcomes

Through this universal exploratory data analysis and baseline modeling experiment, the following core skills were acquired:

1. Mastered key operations in `NumPy`, `Pandas`, `SciPy`, `Scikit-Learn`, `Matplotlib`, and `Seaborn` for data manipulation and visualization.
2. Constructed an automated, reusable EDA pipeline (`perform_EDA`) capable of handling structured tabular, text, and computer vision datasets.
3. Implemented domain-appropriate data cleaning strategies including zero replacement for medical features, log transformation for right-skewed regression labels, and TF-IDF extraction for unstructured text.

4. Evaluated supervised classification and regression algorithms using accuracy, precision, recall, confusion matrices, MAE, RMSE, and R^2 metrics.

11 Conclusion

In this experiment, five distinct machine learning datasets were systematically analyzed, preprocessed, and modeled. Decision Tree achieved 100% accuracy on Iris, SVM achieved 97.58% on Email Spam and 96.35% on MNIST digits, Logistic Regression led Diabetes prediction with 75.32% accuracy, and Random Forest Regressor achieved an R^2 score of 0.3228 on Loan Amount prediction. The experiment demonstrates the vital role of thorough Exploratory Data Analysis and tailored preprocessing in maximizing model performance.

12 References

[1] Scikit-learn Documentation. *Supervised Learning Preprocessing Modules*. https://scikit-learn.org/stable/

[2] Pandas Development Team. *Pandas API Reference User Guide*. https://pandas.pydata.org/docs/

[3] Kaggle Datasets Repository. *Predict Loan Amount, Email Spam SMS, and MNIST CSV*. https://www.kaggle.com/datasets

[4] UCI Machine Learning Repository. *Iris and Pima Indians Diabetes Datasets*. https://archive.ics.uci.edu/ml/